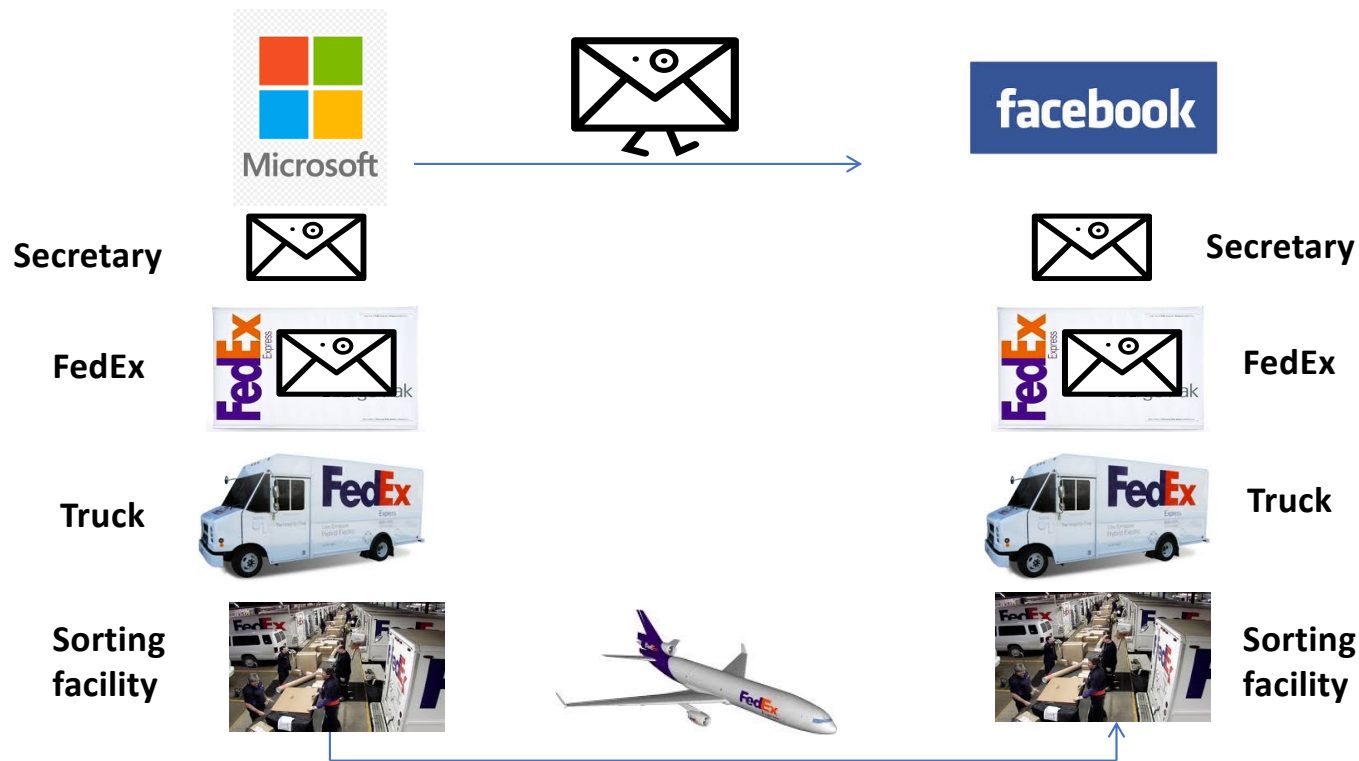


Overview/Roadmap

- What *is* the Internet?
- What *is* a protocol?
- Network edge: hosts, access network, physical media
- Network core: packet/circuit switching, internet structure
- Performance: transmission rate, loss, delay, throughput
- Protocol layers, service models

A simple daily-life example



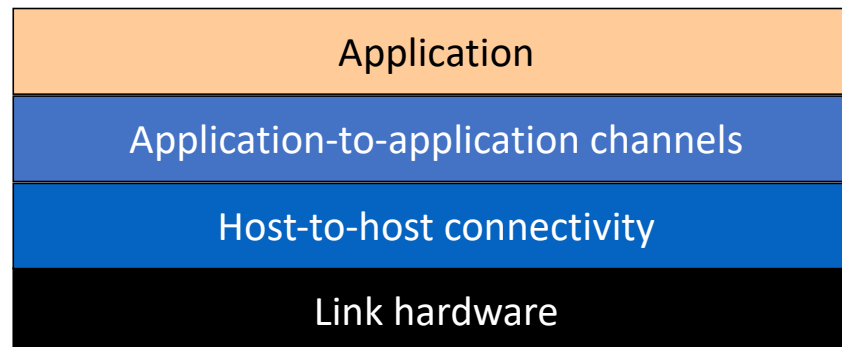
Layering: A Modular Approach

- **Divide and conquer**

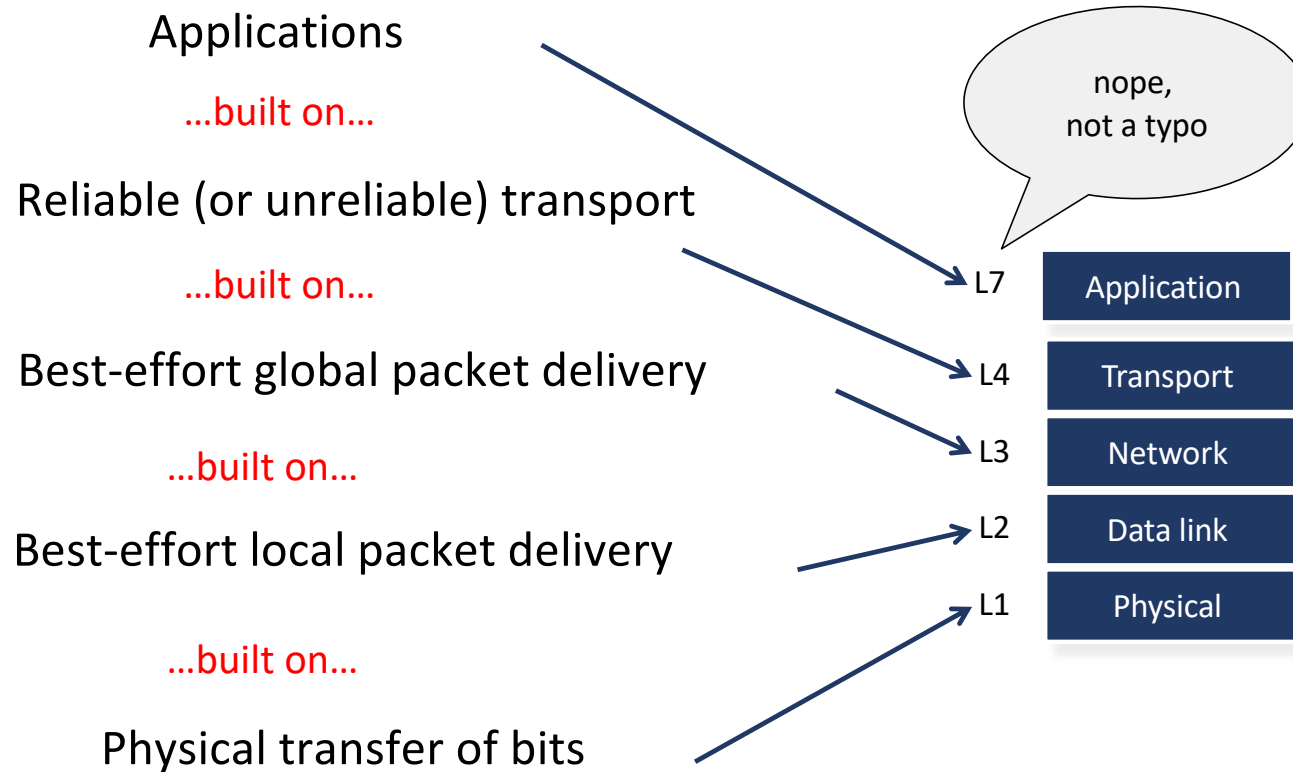
- Each layer relies on services from layer below
- Each layer exports services to layer above

- Interface between layers defines interaction

- Hide implementation details
- Can change layers without affecting other layers

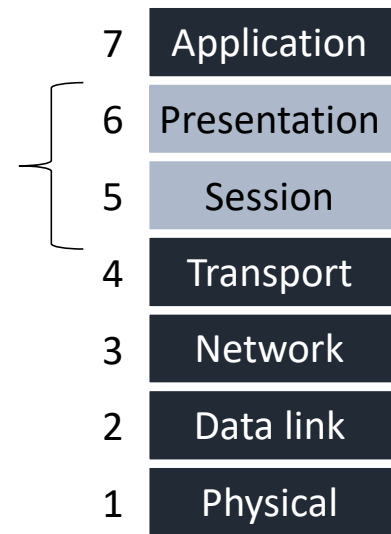


In the Context of Internet



In the Context of Internet

The Open Systems Interconnect (OSI) model developed by the ISO included two additional layers that are often implemented as part of the application



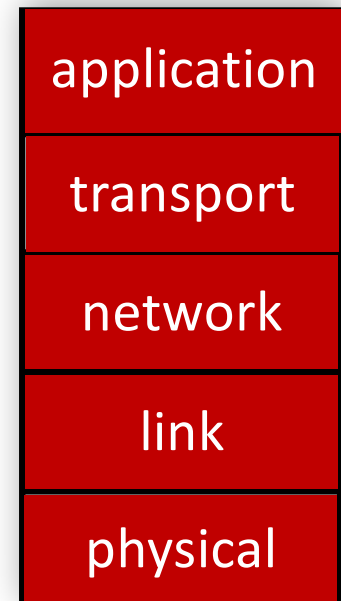
Protocols and Layers



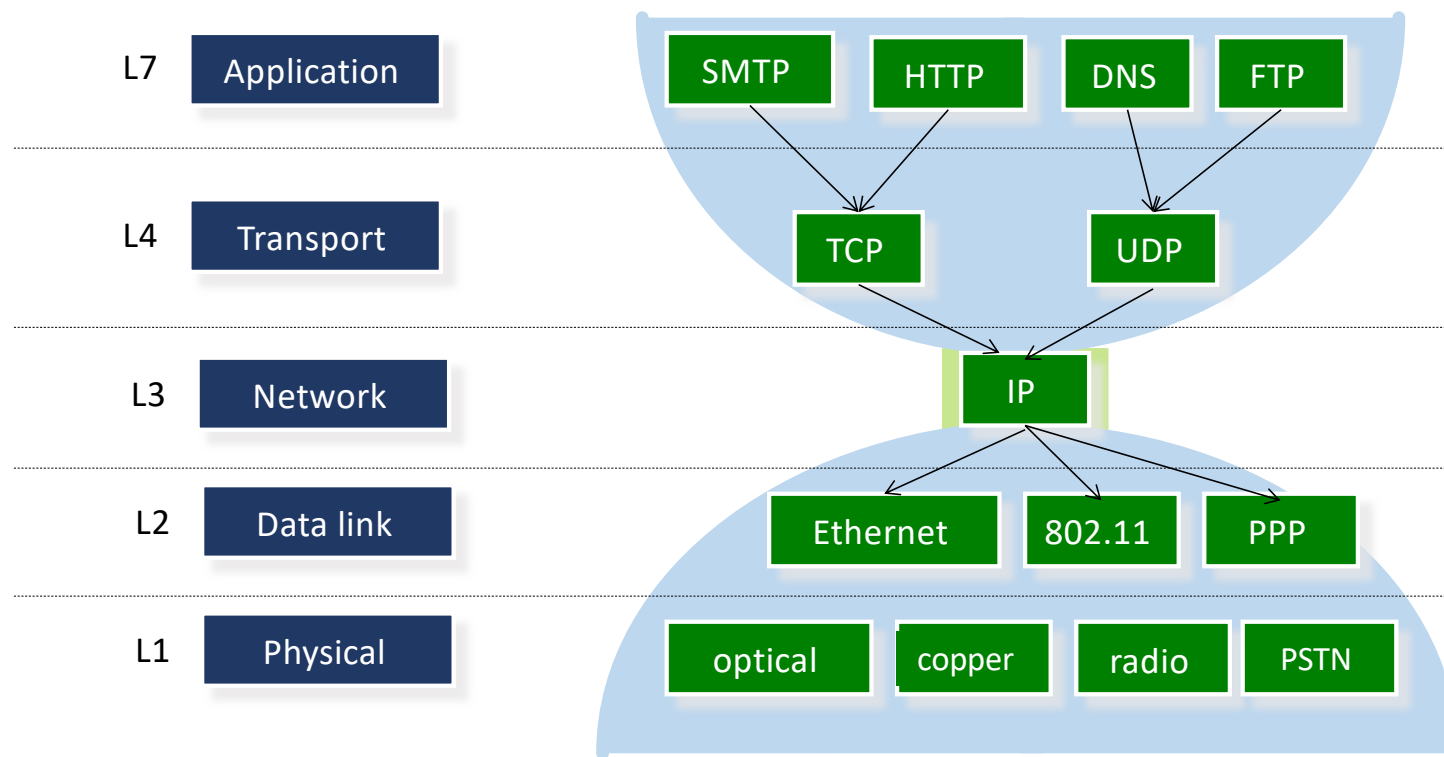
Communication between peer layers on different systems is defined by **protocols**

Layered Internet protocol stack

- *application*: supporting network applications
 - HTTP, IMAP, SMTP, DNS
- *transport*: process-process data transfer
 - TCP, UDP
- *network*: routing of datagrams from source to destination
 - IP, routing protocols
- *link*: data transfer between neighboring network elements
 - Ethernet, 802.11 (WiFi), PPP
- *physical*: bits “on the wire”

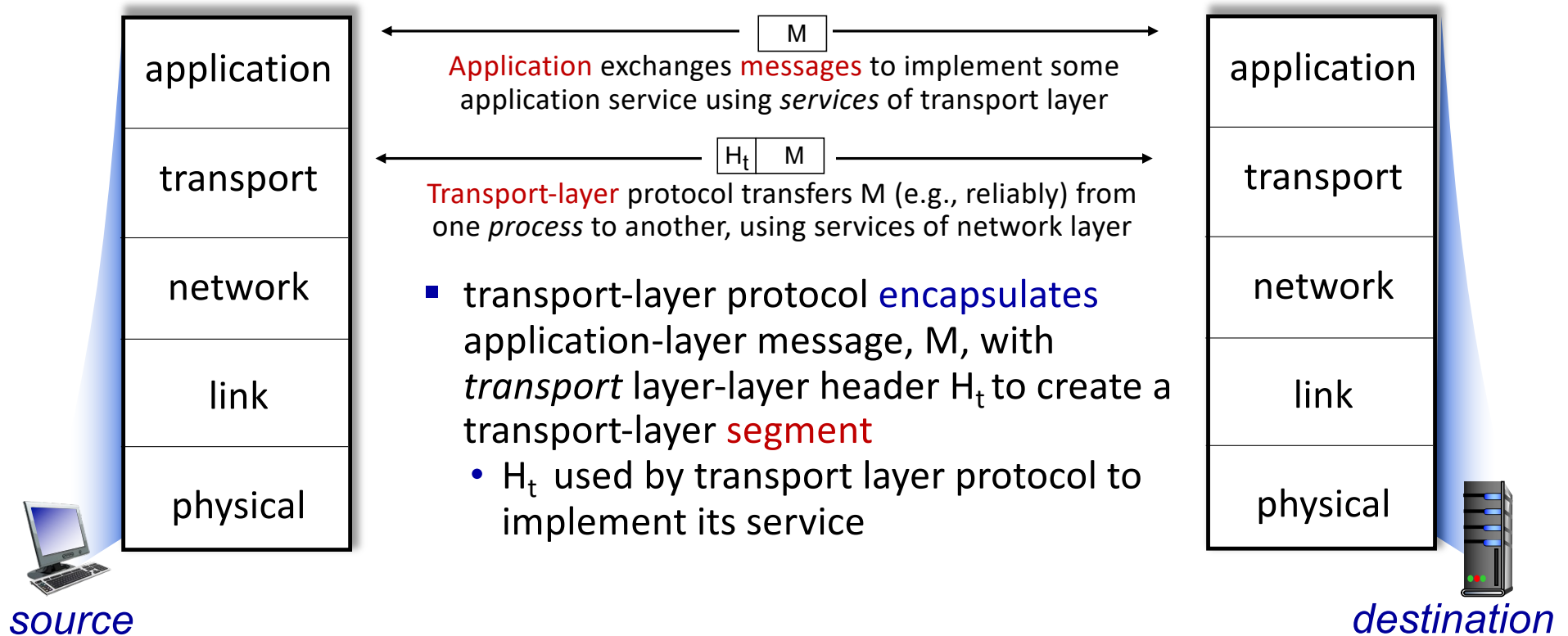


Protocols at Different Layers

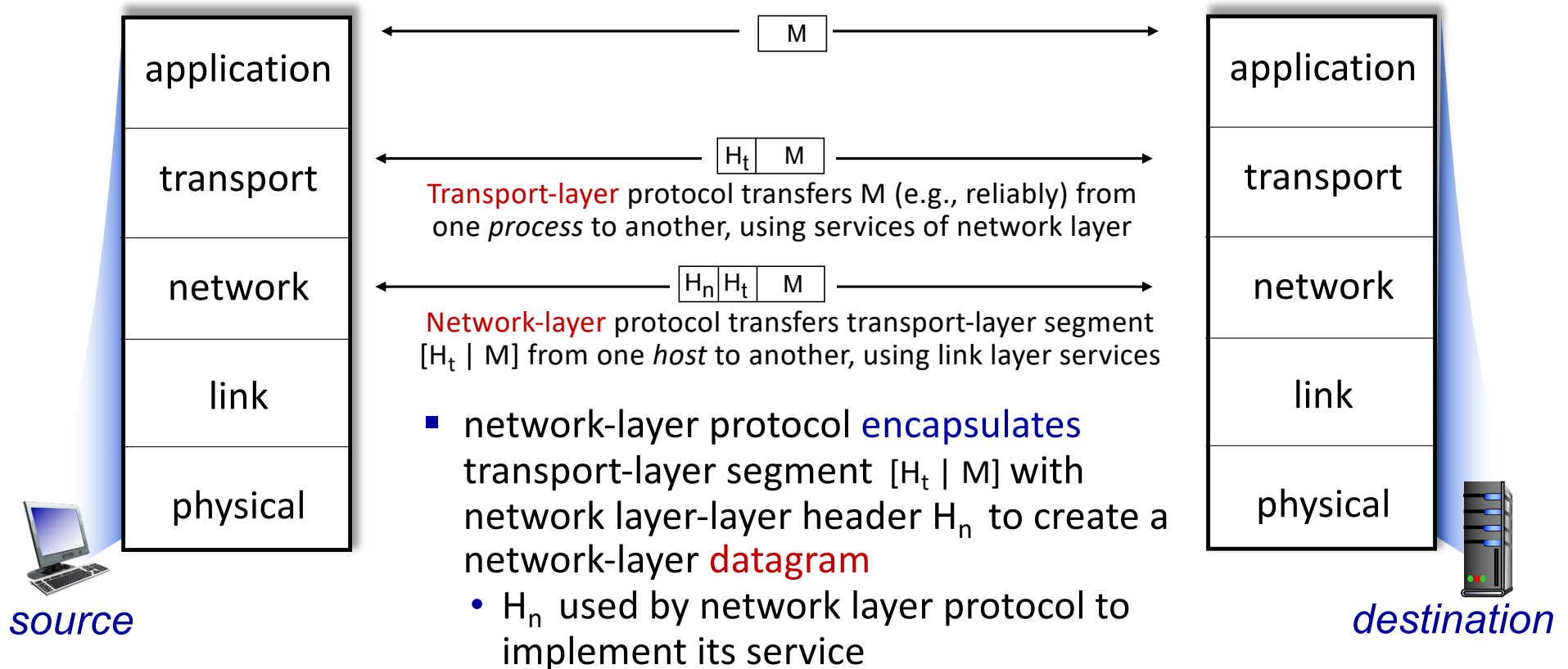


The hourglass model

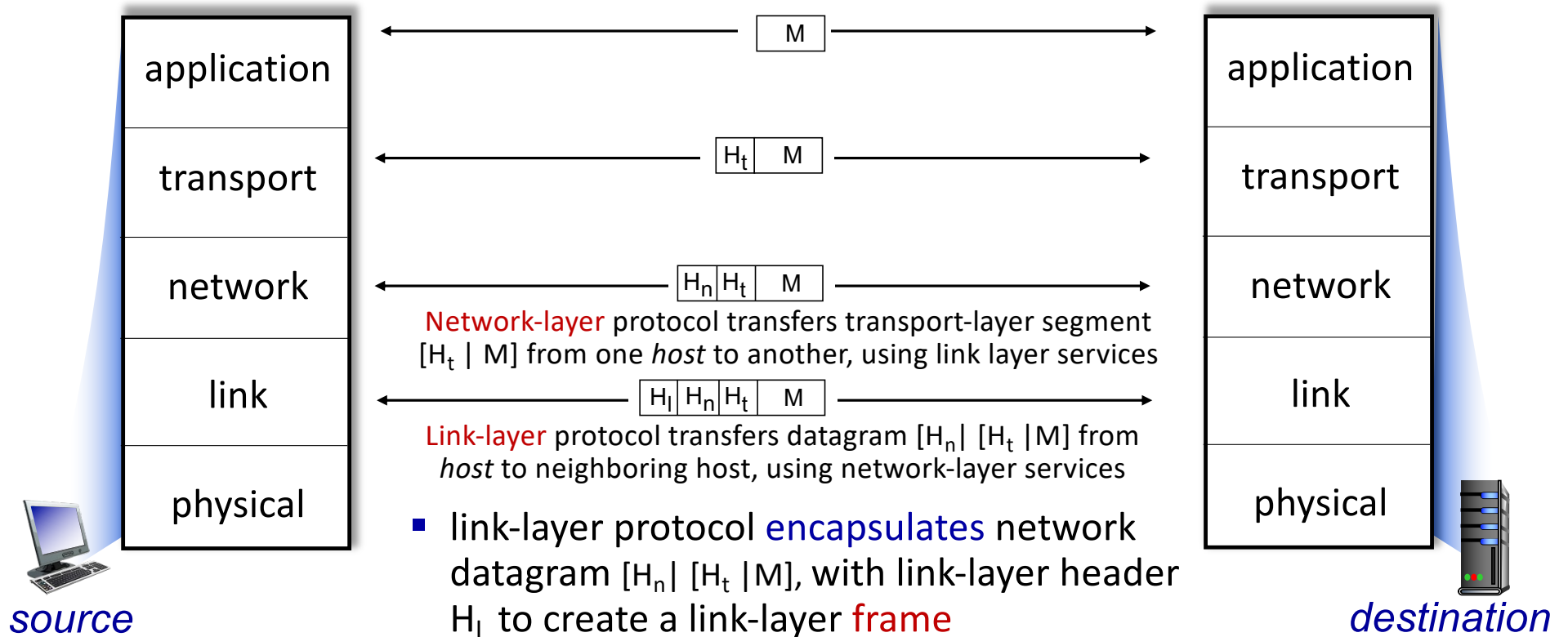
Services, Layering and Encapsulation



Services, Layering and Encapsulation

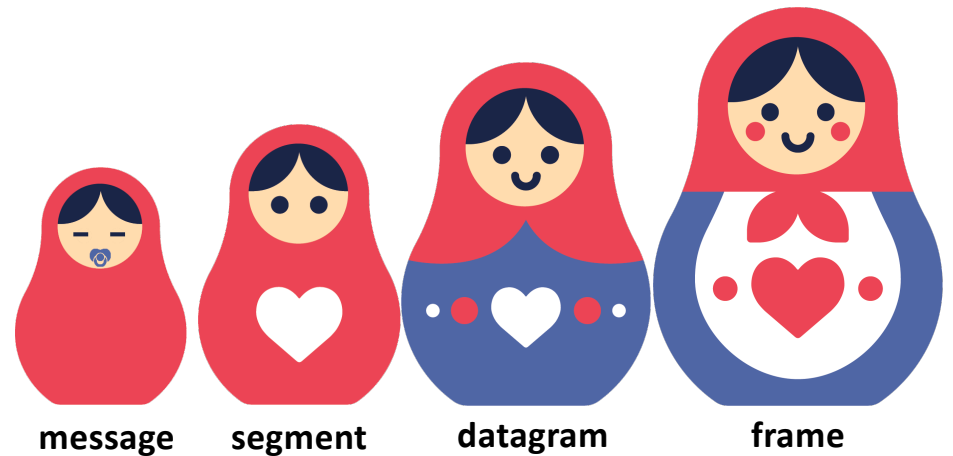


Services, Layering and Encapsulation

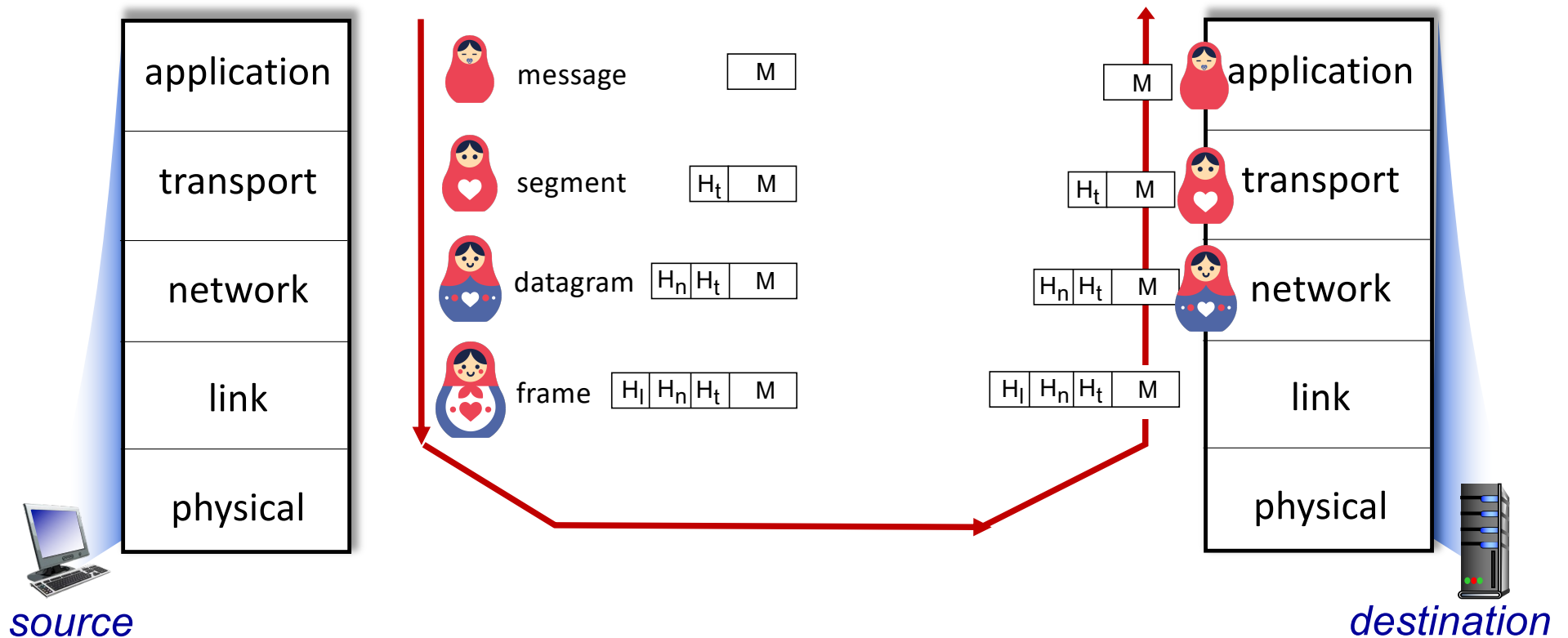


Encapsulation

Matryoshka dolls (stacking dolls)

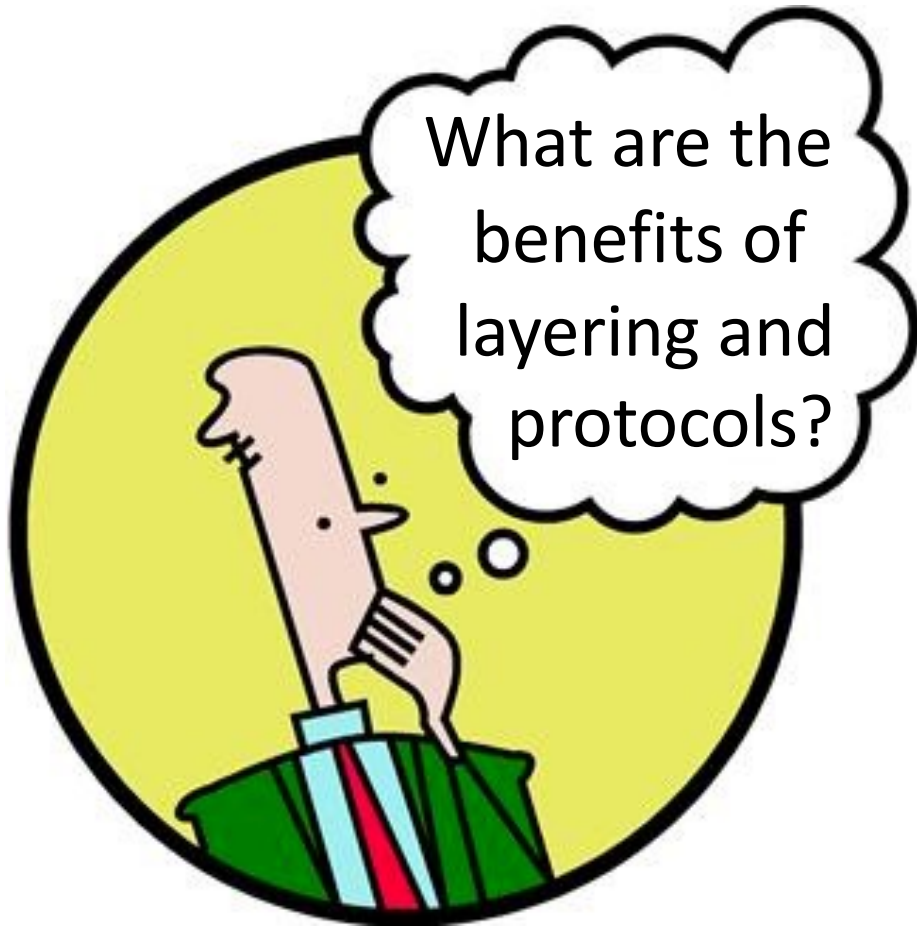


Services, Layering and Encapsulation



What Gets Implemented on the End-host?

- All the layers!
- **Application**
 - User space: mail, game, browser...
- **Transport and network**
 - Typically part of the operating system
- **Data link**
 - Often written by the network interface vendor
- **Physical**
 - Hardware: network interface card and link



- **Interoperability**

- Diverse implementation of many technologies
- They can all talk because they use the same protocols

- **Abstraction and reuse**

- Build atop a standard interface
- Hide the details

- **Stimulating innovation**

- Innovation at each layer can proceed in parallel



- High-level layers may duplicate low-level functionalities
 - Error recovery to retransmit lost data
- May hurt performance
 - Hide details about what is going on
- May duplicate information as well
 - Timestamps, maximum transmission unit (MTU) size
- Some layers are not cleanly separated
 - Inter-layer dependencies for performance reasons
- Header overhead
 - Can get really big...

How to place network functionality

- **End-to-end** principle
 - Highly influential paper: [*End-to-End Arguments in System Design*](#), by Saltzer, Reed, and Clark, 1984
- Some application requirements can only be correctly implemented end-to-end
 - e.g., reliability, security
- Example: How would you ensure a reliable file transfer?

End-to-End Principle

- Implementing functionality in the network
 - Does not reduce host implementation complexity
 - Does increase network complexity
 - Can increase delay and overhead, especially for applications that do not need the functionality
- But, implementing it in the network can improve performance
 - Say a very lossy link
- **Takeaway:** *Implement a functionality in the network only if sufficient, necessary, and useful*



CSEE4119: Computer Networks

Application Layer

Xia Zhou
Department of Computer Science
Columbia University

Slides credited to J.F Kurose and K.W. Ross and their textbook "Computer Networking: A Top-Down Approach"

Application layer: overview

- Principles of network applications
- Web and HTTP
- The Domain Name System DNS
- P2P applications
- video streaming and content distribution networks
- socket programming with UDP and TCP

Network apps

- social networking
- Web
- text messaging
- e-mail
- multi-user network games
- streaming stored video (YouTube, Hulu, Netflix)
- P2P file sharing
- voice over IP (e.g., Skype)
- real-time video conferencing (e.g., Zoom)
- Internet search
- remote login
- access to large AI models
-

Q: *your* favorites?

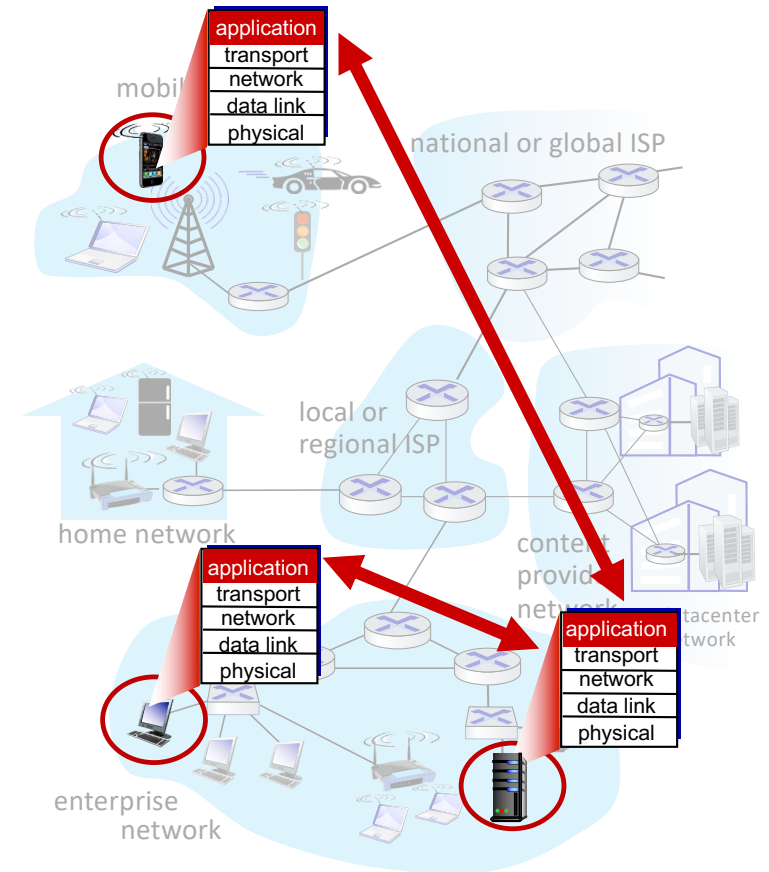
Creating a network app

write programs that:

- run on (different) end systems
- communicate over network
- e.g., web server software communicates with browser software

no need to write software for network-core devices

- network-core devices do not run user applications
- applications on end systems allows for rapid app development, propagation



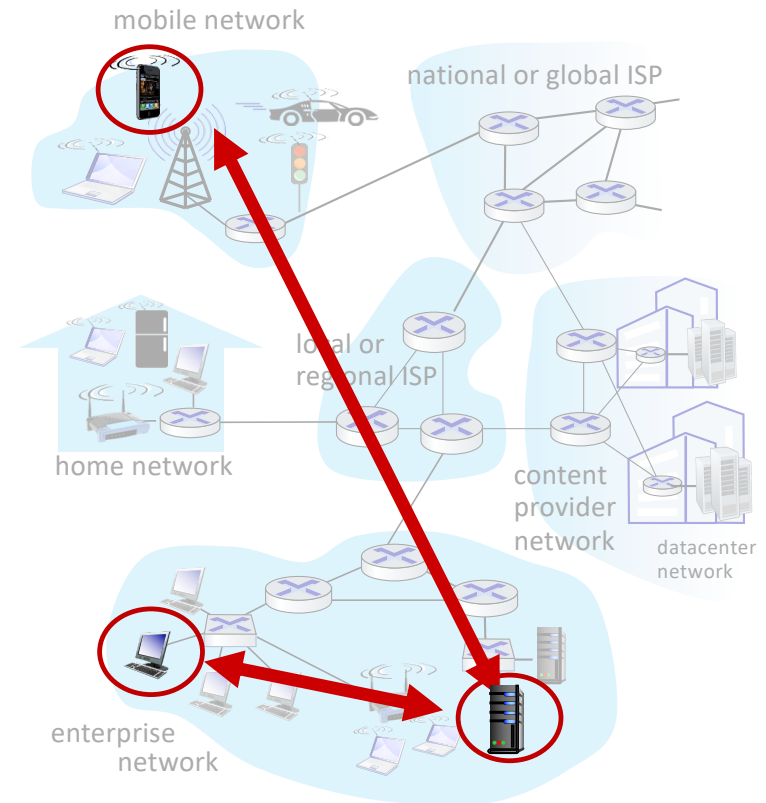
Client-server paradigm

server:

- always-on host
- permanent IP address
- often in data centers, for scaling

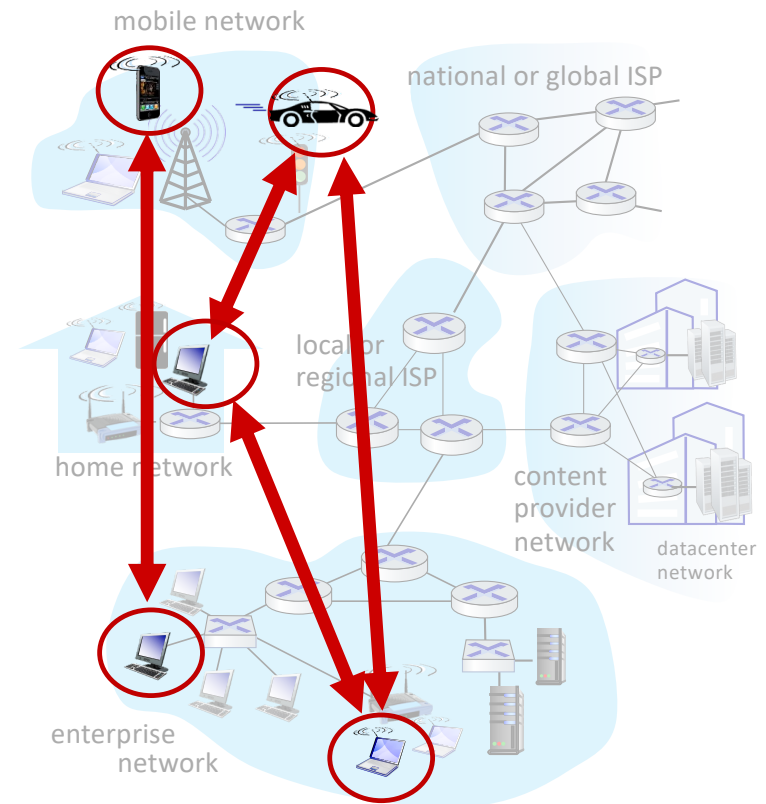
clients:

- contact, communicate with server
- may be intermittently connected
- may have dynamic IP addresses
- do *not* communicate directly with each other
- examples: HTTP, IMAP, FTP



Peer-peer architecture

- *no* always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
 - *self scalability* – new peers bring new service capacity, as well as new service demands
- peers are intermittently connected and change IP addresses
 - complex management
- example: P2P file sharing [BitTorrent]



Processes communicating

process: program running within a host

- within same host, two processes communicate using **inter-process communication** (defined by OS)
- processes in different hosts communicate by exchanging **messages**

clients, servers

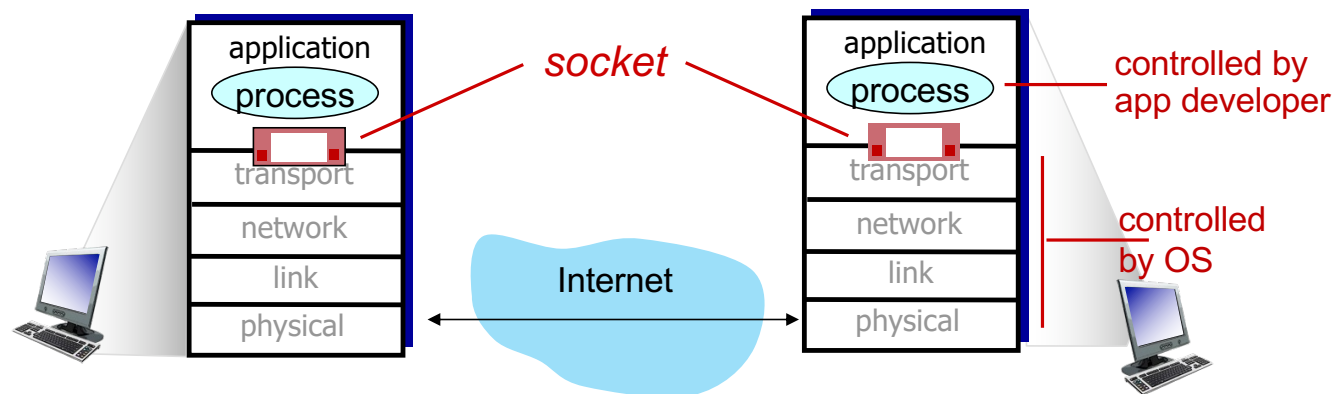
client process: process that initiates communication

server process: process that waits to be contacted

- note: applications with P2P architectures have client processes & server processes

Sockets

- process sends/receives messages to/from its **socket**
- socket analogous to door
 - sending process shoves message out of the door
 - transport layer on other side of door deliver message to socket at receiving process
 - two sockets involved: one on each side



Identifying Receiving Process

- Identifying the receiving end-host
 - Name or address that uniquely identifies the host
 - **IP address (32 bits)**
- Identifying the receiving process in the receiver
 - Host may run many different processes
 - **Destination port (16 bits)**
- Uniquely identifying the traffic between the hosts
 - Two IP addresses and two port numbers
 - Pass them to the underlying transport protocols

An application-layer protocol defines:

- **message type**,
 - e.g., request, response
- **message syntax**:
 - what fields in messages & how fields are delineated
- **message semantics**
 - meaning of information in fields
- **rules** for when and how processes send & respond to messages

open protocols:

- defined in RFCs, everyone has access to protocol definition
- allows for interoperability
- e.g., HTTP, SMTP

proprietary protocols:

- e.g., Skype, Zoom

What transport service does an app need?

data integrity

- some apps (e.g., file transfer, web transactions) require 100% reliable data transfer
- other apps (e.g., audio) can tolerate some loss

timing

- some apps (e.g., Internet telephony, interactive games) require low delay to be “effective”

throughput

- some apps (e.g., multimedia) require minimum amount of throughput to be “effective”
- other apps (“elastic apps”) make use of whatever throughput they get

security

- encryption, data integrity, ...

Transport service requirements: common apps

application	data loss	throughput	time sensitive?
file transfer/download	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5Kbps-1Mbps video:10Kbps-5Mbps	yes, 10's msec
streaming audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	Kbps+	yes, 10's msec
text messaging	no loss	elastic	yes and no

Internet transport protocols services

TCP service:

- *reliable transport* between sending and receiving process
- *flow control*: sender won't overwhelm receiver
- *congestion control*: throttle sender when network overloaded
- *connection-oriented*: setup required between client and server processes
- *does not provide*: timing, minimum throughput guarantee, security

UDP service:

- *unreliable data transfer* between sending and receiving process
- *does not provide*: reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup.

Q: why bother? *Why* is there a UDP?

Internet applications, and transport protocols

application	application layer protocol	transport protocol
file transfer/download	FTP [RFC 959]	TCP
e-mail	SMTP [RFC 5321]	TCP
Web documents	HTTP [RFC 7230, 9110]	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], or proprietary	TCP or UDP
streaming audio/video	HTTP [RFC 7230], DASH	TCP
interactive games	WOW, FPS (proprietary)	UDP or TCP

Securing TCP

Vanilla TCP & UDP sockets:

- no encryption
- cleartext passwords sent into socket traverse Internet in cleartext (!)

Transport Layer Security (TLS)

- provides encrypted TCP connections
- data integrity
- end-point authentication

TLS implemented in application layer

- apps use TLS libraries, that use TCP in turn
- cleartext sent into “socket” traverse Internet *encrypted*
- more: Chapter 8

Quiz Time

- All the layers get implemented on end hosts
- **Application**
 - User space: mail, game, browser...
- **Transport and network**
 - Typically part of the operating system
- **Data link**
 - Often written by the network interface vendor
- **Physical**
 - Hardware: network interface card and link

